

Boldscholars/Lastprep

Machine Learning

A brief Introduction

INTRODUCTION TO MACHINE LEARNING

Topics Covered:

Learning Problems – Perspectives and Issues

A brief introduction to Machine Learning

Supervised Learning, Unsupervised Learning, Reinforcement Learning

Concept Learning – Version Spaces and Candidate Eliminations

Inductive bias, Decision Tree learning.

Learning Problems – Perspectives and Issues

A **learning problem** is defined as a problem where a computer program **improves its performance** at a task through **experience**.

According to **Tom Mitchell**:

A computer program is said to learn from experience E with respect to some task T and performance measure P , if its performance at task T , as measured by P , improves with experience E .

Components of a Learning Problem

1. **Task (T)** – What the system should do

Example: Classify emails as spam or not spam

2. **Experience (E)** – Data or past examples

Example: Previously labelled emails

3. **Performance Measure (P)** – How success is measured

Example: Accuracy of classification

For example, a computer program that learns to play checkers might improve its performance as measured by its ability to win at the class of tasks involving playing checkers games, through experience obtained by playing games against itself.

In general, to have a well-defined learning problem, we must identify these three features:

The class of tasks,

The measure of performance to be improved, and

The source of experience.

There are several factors affecting the performance:

- **Types of training** provided

- The form and extent of any initial **background knowledge**
- The **type of feedback** provided
- The **learning algorithms** used

Two important factors:

- Modeling
- Optimization

Examples of Learning Problems

A checkers learning problem:

Task T: Playing checkers

Performance measure P: Percent of games won against opponents

Training experience E: Playing practice games against itself

A robot driving learning problem:

Task T: Driving on public four-lane high ways using vision sensors

Performance measure P: Average distance traveled before an error (as judged by human overseer) 0

Training experience E: A sequence of images and steering commands recorded while observing a human driver

To better filter emails as spam or not

Task T -:Classifying emails as spam or not

Performance Measure P: The fraction of emails accurately classified as spam or not spam

Experience E: Observing you label emails as spam or not spam

Handwriting Recognition Problem

Task T- Acknowledging handwritten words within portrayal

Performance Measure P- Percent of words accurately classified

Experience E - A directory of handwritten words with given classifications

Fruit Prediction Problem

Task T- Forecasting different fruits for recognition

Performance Measure P - Able to predict maximum variety of fruits

Experience E - Training machine with the largest datasets of fruits images

Face Recognition Problem

Task T- Predicting different types of faces

Performance Measure P- Able to predict maximum types of faces

Experience E- Training machine with maximum amount of datasets of different face images

Automatic Translation of documents

Task T - Translating one type of language used in a document to other language

Performance Measure P- Able to convert one language to other efficiently

Experience E - Training machine with a large dataset of different types of languages

While Machine learns a task, it may come across the following common learning problems:

Inadequate training data: This is a common problem, where the data used to train the algorithm is insufficient, inconsistent, or unclean.

For example, if you want to create an algorithm to distinguish spam from safe emails, you'll need examples of both.

Under fitting: This occurs when the data doesn't clearly link the input and output variables.

Over fitting: This occurs when the model is trained with too much data, which can negatively impact performance.

Delayed implementation: Machine learning models can take a long time to implement due to data overload, slow programs, and other factors.

Bad recommendations: The model may produce bad recommendations (irrelevant content)

Algorithmic bias: This can trigger costly errors and lead projects and organizations in the wrong direction.

The following ways may handle the above mentioned issues

- Using high-quality, representative data
- Monitoring the model regularly
- Detecting and mitigating bias
- Updating and monitoring data regularly
- Investing in training and education

Perspectives of Learning

The following are the Different ways of understanding or approaching machine learning algorithms

Explainability: Perspective is important for developing machine learning models that are interpretable and explainable. As models become more complex, it can be difficult to understand how they make decisions.

Decision-making: Machine learning can be viewed as automated decision-making at scale. This perspective can be useful when considering machine learning from an economics or fairness perspective.

User experience: A user experience perspective can be considered when developing machine learning.

Infrastructure: Machine learning models need to be considered from an infrastructure perspective.

Data: Data is essential for machine learning.

Tree-based approaches: Tree-based approaches can be used to solve classification and regression problems. In these approaches, data is partitioned into smaller subsets, similar to the structure of a tree.

Issues in Machine Learning

The Field of Machine Learning concerned with answering questions as follow

1. Which learning algorithms should we use, and when do they work best

There are many machine learning algorithms (Decision Trees, KNN, SVM, Neural Networks, etc.). We need to know

- Which algorithm to choose
- For which type of problem
- Under what conditions it learns correctly when enough data is available

Example

- For **email spam detection**, Naïve Bayes works well
- For **image recognition**, Deep Neural Networks work better
- For **small datasets**, Decision Trees may perform better than Deep Learning

Different problems need different algorithms.

2. How much training data is enough

Machine learning models learn from examples.

- How many examples are needed to learn reliably?
- How confident can we be in the learned model?

Example:

To recognize handwritten digits

With **10 images**, the model performs poorly

With **10,000 images**, performance improves

A simple model needs less data. A complex model (like Deep Learning) needs more data.

More data usually improves learning, but the required amount depends on model complexity.

3. How can prior knowledge help learning

Prior knowledge means **information already known** before learning starts. This knowledge can guide learning and reduce errors.

- In **face recognition**, knowing that “eyes are usually above the nose” helps the model
- In **agriculture**, knowing that “temperature affects crop yield” helps prediction models

Even if prior knowledge is **not perfectly correct**, it can still improve learning.

Prior knowledge makes learning faster and more accurate.

4. How should we choose the next best training example

Instead of using random data, the learner can **select useful examples** to learn faster. This idea is called **active learning**.

- A student learns better by focusing on **difficult questions** rather than easy ones
- A model trained to classify images may ask for labels of **uncertain images only**

Smart selection of training data reduces learning time and complexity.

5. How to reduce a learning problem into simpler function approximation tasks

Complex problems can be broken into **smaller functions** that are easier to learn. For **speech recognition**:

1. Detect sound features
2. Recognize phonemes
3. Form words
4. Understand sentences

Instead of learning everything at once, the system learns step by step. Breaking problems into smaller parts improves learning.

6. What functions should the system learn and can this choice be automated

- What exactly should the model try to learn?
- Can the system decide this automatically?

In image classification:

- Earlier systems used **hand-crafted features**
- Modern deep learning **automatically learns features** like edges, shapes, and objects

Modern ML systems can automatically decide what functions to learn.

7. How can the learner change its representation to learn better?

Representation means **how data is described** (features).
The learner can improve performance by changing how data is represented.

- Instead of raw crop images:
 - Use **NDVI values** for crop health
- Instead of raw text:
 - Use **word embeddings**

Deep learning models automatically change representations through layers.

Issue	Simple Idea	Example
Algorithm choice	Different algorithms suit different problems	CNN for images
Data size	More data → better learning	10 vs 10,000 samples
Prior knowledge	Existing knowledge helps	Weather affects crops
Training selection	Choose useful examples	Active learning
Problem decomposition	Break complex tasks	Speech recognition steps
Function choice	Decide what to learn	Auto feature learning
Representation	Change data format	NDVI instead of raw images

Machine Learning is not just about training models, but about choosing the right algorithm, data, knowledge, representation, and learning strategy to achieve good generalization.

Example: Crop Yield Loss Prediction After Heat waves and Pest Attacks

Problem Statement:- An agricultural department wants to **predict crop yield loss** in advance using weather data, satellite images, and pest information, so that farmers can take preventive action.

1. Choice of Learning Algorithm

Which algorithm works best?

- **Random Forest** → for tabular weather and pest data
- **CNN** → for satellite images (crop health)
- **ConvLSTM** → for time-series satellite + weather data

ConvLSTM is chosen because crop stress develops over **time and space**.

Different algorithms are suitable for different data types.

2. How Much Training Data Is Sufficient

How much data is enough?

- 1 year of data → poor predictions
- 5–10 years of historical weather, pest, and yield data → reliable predictions

More seasons help the model understand **normal vs extreme heatwave patterns**.

More data increases confidence in predictions.

3. Use of Prior Knowledge

Can prior knowledge help?

Agricultural experts know that

- High temperature + low rainfall → crop stress
- Certain pests appear after heatwaves

This knowledge is used to

- Select relevant features (temperature, humidity, pest index)
 - Initialize model constraints
- Even approximate agricultural knowledge improves learning.

4. Choosing the Next Best Training Example

How to select useful training samples?

The system focuses on:

- Regions with **unusual yield drops**
- Years with **extreme heatwaves**

Instead of learning from normal years repeatedly, the model learns more from **disaster-affected seasons**. Smart data selection speeds up learning.

5. Reducing the Problem into Function Approximation Tasks

How to simplify learning?

The problem is divided into:

1. Crop health estimation from satellite images
2. Pest outbreak prediction
3. Weather stress analysis
4. Final yield loss estimation

Each task is learned separately and then combined.

Breaking complex problems into smaller functions improves accuracy.

6. What Functions Should the System Learn? Can This Be Automated?

What exactly should be learned?

- Traditional methods used **manually designed features**
- Deep learning automatically learns
 - Vegetation stress patterns
 - Pest damage indicators

CNN automatically learns crop damage features from images. Modern ML automates feature learning.

7. Improving Representation Automatically

How to change data representation?

- Raw satellite images → converted to **NDVI maps**
- Weather readings → transformed into **stress indices**
- Deep layers refine representations automatically

Instead of raw temperature values, the model learns **heat stress representations**.

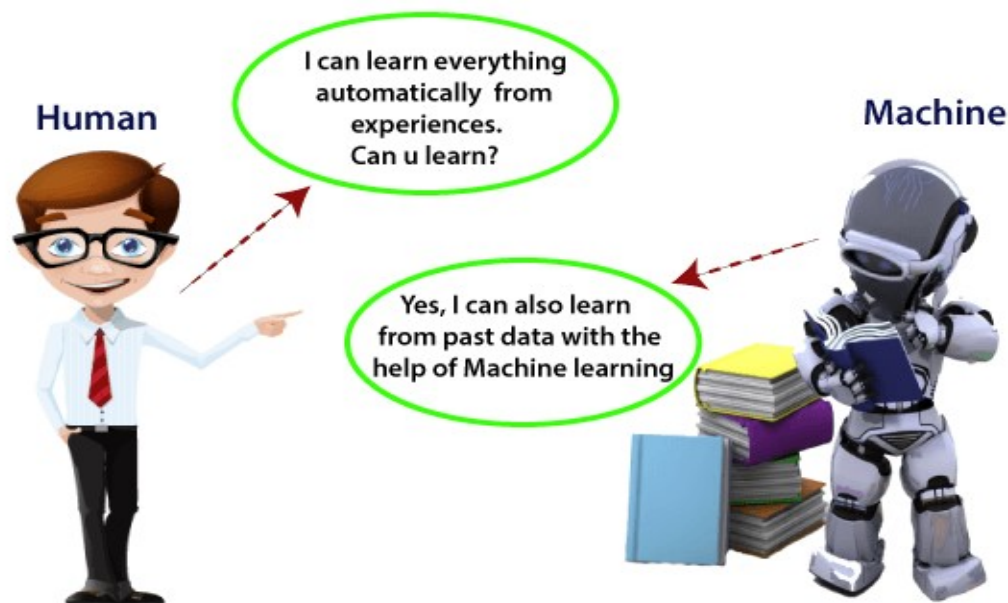
Better representations lead to better predictions.

This single agricultural application addresses **all machine learning issues**:

- Algorithm selection
- Data sufficiency
- Use of domain knowledge
- Smart data selection
- Problem decomposition
- Automated function learning
- Representation improvement

Crop yield loss prediction demonstrates how machine learning systems must carefully select algorithms, data, knowledge, representations, and learning strategies to achieve accurate real-world performance.

Introduction to Machine Learning:

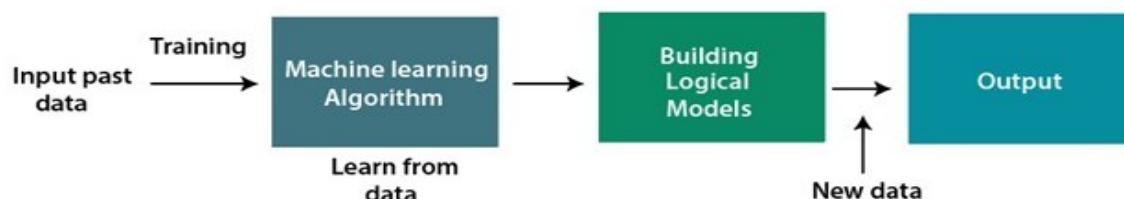


Machine Learning (ML) is a subset of artificial intelligence (AI) that focuses on the development of algorithms and statistical models that enable computers to learn and make decisions from data without being explicitly programmed for each task. Essentially, it involves teaching machines to recognize patterns, make predictions, and improve from experience.

How does Machine Learning work

A Machine Learning system **learns from historical data, builds the prediction models, and whenever it receives new data, predicts the output for it.** The accuracy of predicted output depends upon the amount of data, as the huge amount of data helps to build a better model which predicts the output more accurately.

Suppose we have a complex problem, where we need to perform some predictions, so instead of writing a code for it, we just need to feed the data to generic algorithms, and with the help of these algorithms, machine builds the logic as per the data and predict the output. Machine learning has changed our way of thinking about the problem.



The Process of Machine Learning involves,

1. Data as an Input:

- **Raw Data:** This is the initial data collected from various sources. It can be

structured (like tables in databases) or unstructured (like text, images, and videos).

- **Pre-processing:** This involves cleaning and transforming the raw data into a format suitable for analysis.
- It includes handling missing values, normalizing data, and encoding categorical variables.

2. Training Phase:

- **Algorithm Selection:** Choose an appropriate machine learning algorithm based on the type of problem (classification, regression, clustering, etc.).
- **Model Training:** Feed the pre processed data into the algorithm. The model learns from the data by adjusting its parameters to minimize errors.
- This process involves splitting the data into training and validation sets to tune the model's performance.

3. Evaluation Phase:

- **Testing the Model:** Assess the model's performance using a separate test set that was not seen during training. Common evaluation metrics include accuracy, precision, recall, F1 score, and mean squared error, depending on the problem type.
- **Cross-Validation:** A technique to ensure the model generalizes well to unseen data by splitting the data into multiple subsets and training/testing the model on different combinations of these subsets.

4. Prediction Phase:

- **Deployment:** Once the model is trained and evaluated, it is deployed to make predictions on new, unseen data.
- **Continuous Learning:** In some applications, the model continuously learns from new data to improve its accuracy and adapt to changes over time.

Why Machine Learning / Need for Machine Learning

Complexity of Tasks: Machine learning is needed because it can handle tasks that are too complex for humans to implement directly.

Data Accessibility: Humans have limitations in manually accessing and processing large amounts of data, so machine learning helps by efficiently handling vast volumes of data.

Training and Exploration: Machine learning algorithms explore data, analyze patterns, and construct models by being trained on large datasets.

Automated Predictions: Trained machine learning algorithms can automatically predict outputs or make decisions based on new, unseen data.

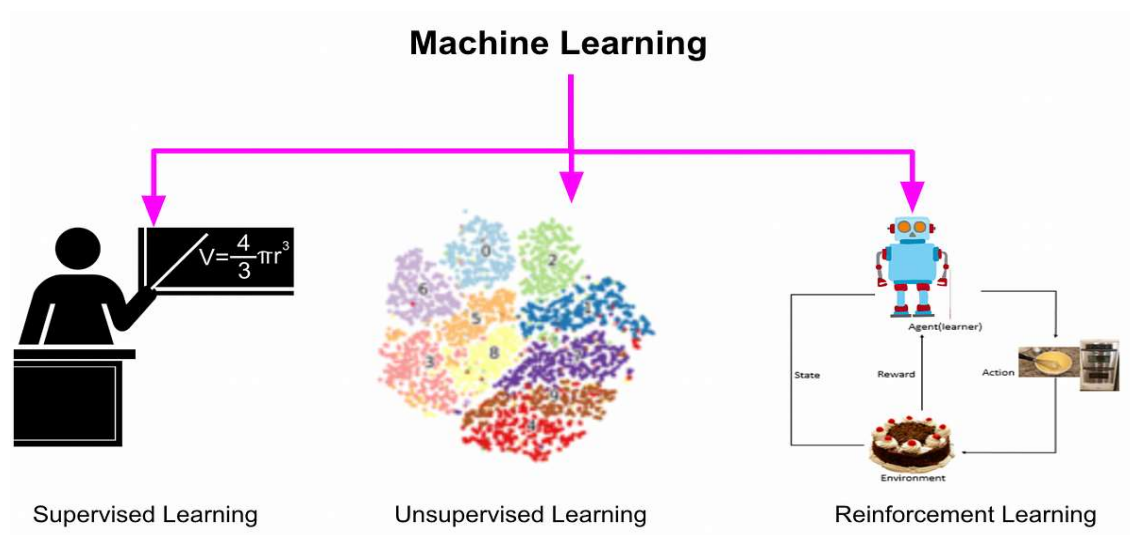
Time and Cost Savings: Machine learning can save time and money by automating complex tasks and streamlining processes.

Use Cases: Machine learning is used in various domains, such as self-driving cars, cyber fraud detection, face recognition, and personalized recommendations by companies like Netflix and Amazon.

Applications

- Email spam detection
- Face recognition
- Recommendation systems
- Medical diagnosis
- Self-driving cars
- Face detection
- Object detection and recognition
- Image segmentation
- Multimedia event detection
- Economical and commercial usage
- Natural Language Processing
- Learning to drive vehicles
- Game Playing and many more....

Classification of Machine Learning



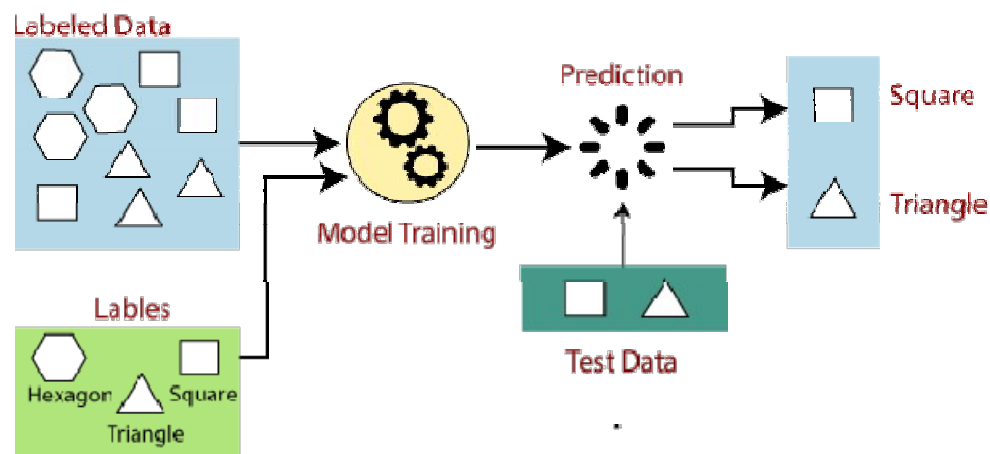
Machine learning can be classified into three types

Supervised learning, Unsupervised learning and Reinforcement learning

Supervised Learning

Supervised machine learning is a technique where a machine is trained using a **labelled dataset**, meaning each input already has a known output. During training, the machine learns the relationship between inputs and their corresponding outputs. After training, it uses this knowledge to predict outputs for new, unseen data (test data).

It can be compared to teaching with examples, such as showing a computer labelled images of cats and dogs. By learning features like shape, size and color the machine can later identify whether a new image is a cat or a dog. The main goal of supervised learning is to make accurate predictions and it is widely used in applications such as spam detection, image recognition, handwriting recognition and medical diagnosis.



Process

1. **Labelled Data:** The dataset contains input data (shapes) along with their correct labels (square, triangle, hexagon).
2. **Training the Model:** The labelled data is given to the machine, and it learns the features of each shape.
3. **Test Data:** New, unlabelled shapes are provided to the trained model.
4. **Prediction:** The model analyses the test data and predicts the correct label.
5. **Output:** The final result is the identified shape (square or triangle).

Supervised Learning Example:

Dataset: Imagine we have a dataset of images containing pictures of cats and dogs. Each image is labeled as either "cat" or "dog."

Training Phase: During training, we show the machine these images and their corresponding labels. The machine learns to recognize features that distinguish cats from dogs, such as tail shape, eye color, height, etc.

Testing Phase: After completing the training, we give the machine a new image of a cat or a dog that it has never seen before. We ask the machine to identify the object and predict whether it's a cat or a dog.

Prediction: The machine uses what it learned during training to analyze the new image. It checks features like height, shape, color, eyes, ears, tail, etc., and based on the patterns it recognized, it makes a prediction.

Mapping Input and Output: The main goal of supervised learning is to map the input variables (the image's features - x) to the output labels (whether it's a cat or a dog - y).

Real-World Applications:

Risk Assessment: Predicting the likelihood of certain events, such as loan defaults or insurance claims, based on historical data.

Fraud Detection: Identifying false transactions or activities by learning from past data of genuine and fraudulent transactions.

Spam Filtering: Classifying emails as spam or non-spam based on their content, using labeled data of known spam emails.

Categories of Supervised Machine Learning

Supervised machine learning can be classified into two types:

1. Classification
2. Regression

Classification: Classification algorithms are used when the **output is categorical** - meaning the result belongs to a specific class or group such as *Yes/No*, *Male/Female* or *Cat/Dog*. These algorithms learn from labelled data and predict the correct category for new inputs.

Example: If a machine is trained using images labelled as “cat” or “dog,” it can later identify whether a new image is a cat or a dog.

Popular Classification Algorithms:

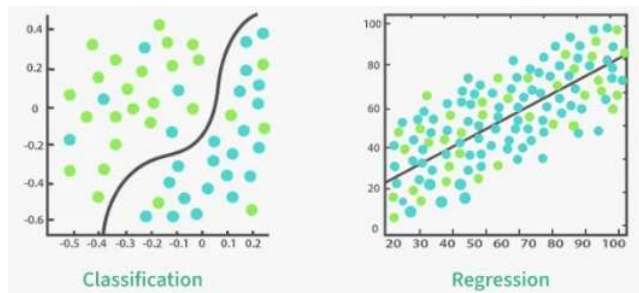
- Decision Tree Algorithm
- Random Forest Algorithm
- Logistic Algorithm

Regression: Regression algorithms are used when the **output is a continuous numerical value**, such as temperature, price, or time. These algorithms analyze past data to learn the relationship between input and output variables and then predict future values.

Example: Based on previous data showing how ice cream sales increase with temperature, a regression model can predict the temperature from sales data.

Popular Regression Algorithms:

- Simple Linear Regression
- Multivariate Regression



Advantages of Supervised Learning

1. Since supervised learning work with the labelled dataset so we can have an exact idea about the classes of objects.
2. These algorithms are helpful in predicting the output on the basis of prior experience.

Disadvantages:

1. These algorithms are not able to solve complex tasks.
2. It may predict the wrong output if the test data is different from the training data.
3. It requires lots of computational time to train the algorithm.

Applications of Supervised Learning

Guessing Songs: Supervised learning helps a music app learn which songs are happy or sad based on their features (like beats and lyrics). Then, when you pick a new song, the app can guess if it's happy or sad.

Plant Disease Detection: Suppose you have pictures of healthy and sick plants. With supervised learning, a program can learn to tell if a plant is healthy or has a disease by studying the differences in the images. When you show it a new plant picture, it can say if the plant is okay or needs help.

Supervised Learning example

Imagine teaching a computer to understand the relationship between the number of apples and the cost in rupees.

Examples: You have some data points

Task: Your goal is to help the computer learn how to calculate the cost in rupees based on the number of apples.

No of Apples	Cost (INR)
2	80
4	160
6	240

Supervised Learning:

Using the examples, the computer learns a formula to calculate the cost.

It might learn:

$$\text{Cost} = 40 * \text{Number of Apples}$$

Now, if you give it a new number of apples, say 5, the computer can predict the

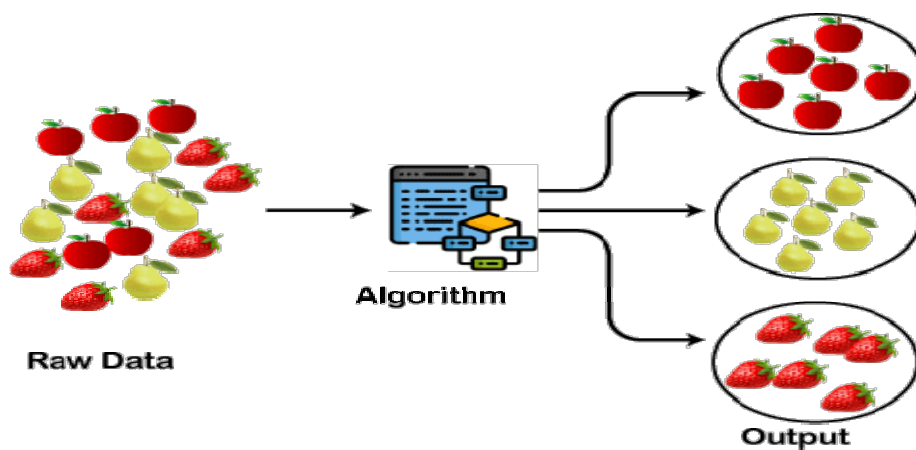
$$\text{Cost} = 40 * 5 = 200$$

So, based on the pattern it learned, the computer predicts that 5 apples would cost 200 INR. This is how supervised learning helps the computer learn and apply calculations to make predictions.

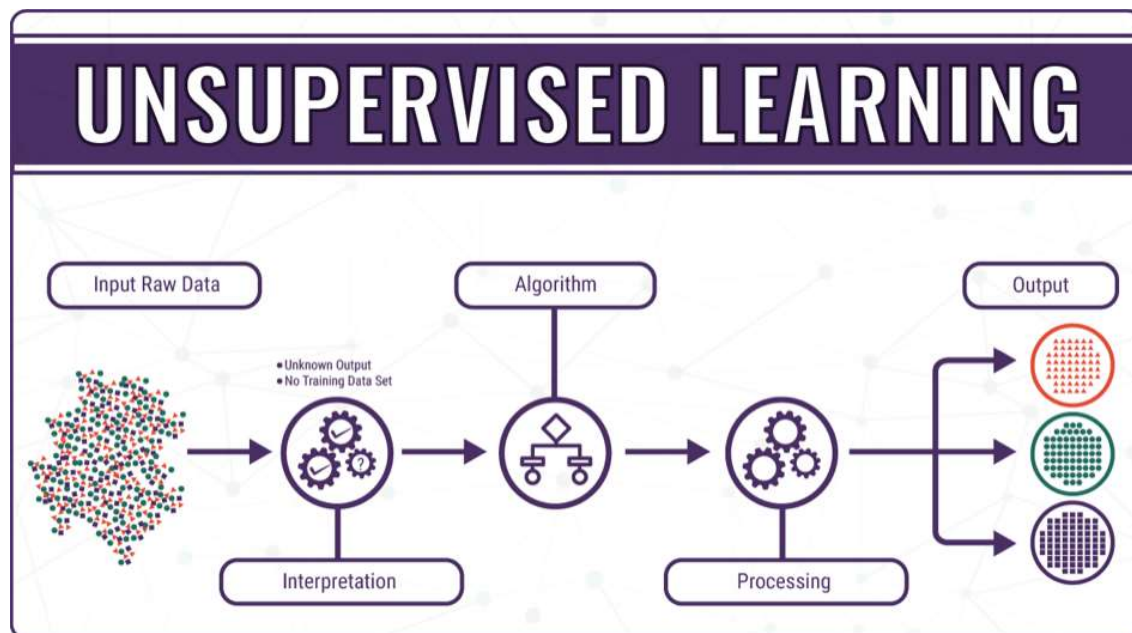
Unsupervised Learning

Unsupervised learning is a type of machine learning where the model is trained using an **unlabelled dataset**, meaning there are no predefined output labels. Unlike supervised learning, the machine learns **without guidance or supervision** and discovers patterns on its own.

In this approach, the model analyses data based on features such as **color, shape, size, or behaviour** and groups similar data points together. For example, if given a collection of different fruits without labels, the model may group round red fruits together and yellow fruits together based on their similarities.



The main goal of unsupervised learning is to **identify hidden patterns, similarities, and structures** in the data. When new data is introduced, the model assigns it to the most appropriate group based on what it has already learned.



Types of Algorithms

1. Clustering 2. Association

Clustering: Clustering is the process of grouping similar data items together based on their features. The machine does not know the labels in advance; it simply finds similarities and forms groups.

Example: Imagine you have many colored pencils and you arrange them by similar colors without knowing their names. Pencils that look alike are placed together. Similarly, in machine learning, clustering groups customers with similar buying behaviour or images with similar patterns.

Association: Association finds relationships between items in data. It identifies which items are frequently used or bought together.

Example: In a store, if customers who buy bread often buy butter, the system finds this relationship. The store can then place these items close together or create offers.

Algorithms

- K-Means
- Hierarchical Clustering
- Apriori Algorithm
- FP-Growth

Un-Supervised Learning example

- Imagine you have a bunch of different-colored balls and you want the computer to organize them into groups based on color..

Examples: You have balls with different colors:

Ball	Color
Ball 1	Red
Ball 2	Blue
Ball 3	Red
Ball 4	Green
Ball 5	Blue
Ball 6	Green

Task: Your goal is to help the computer group the balls based on their colors.

Unsupervised Learning:

Using the examples, the computer discovers patterns and groups balls based on similar colors. It might group them as:

Group 1: Red (Ball 1, Ball 3)

Group 2: Blue (Ball 2, Ball 5)

Group 3: Green (Ball 4, Ball 6)

The computer found these groups without any prior knowledge of colors, it learned from the patterns it observed.

Now, if you show the computer a new ball, say a yellow one, it might assign it to a new group or match it with a similar color group based on its observation.

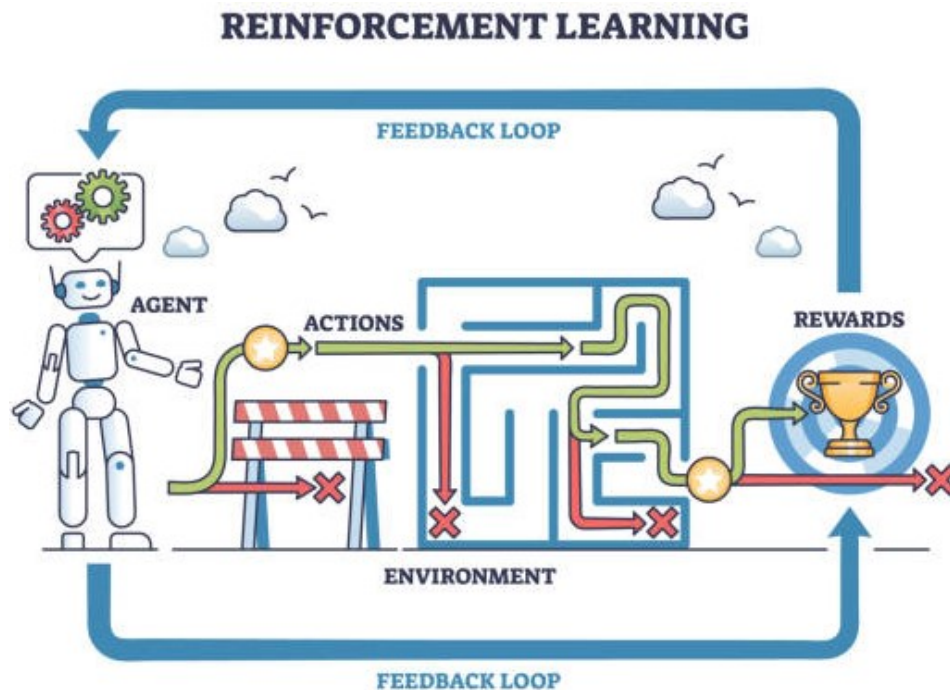
Unsupervised learning helps the computer find similarities and organize data into groups, even when it doesn't have explicit instructions about those groups.

Applications

- Data exploration
- Recommendation systems
- Anomaly detection

Reinforcement Learning

Definition: Reinforcement learning is a type of machine learning in which an **agent learns by interacting with an environment** and improving its actions through **trial and error**. The agent receives **rewards** for correct actions and **penalties** for incorrect ones. Over time, it learns the **best strategy** to maximize total rewards.



How It Works

The agent tries different actions in an environment.

- **Good actions → Reward**
- **Bad actions → Punishment**
- By repeating this process, the agent learns the **optimal path or behaviour**.

Example: A robot navigating a maze gets rewards for moving toward a goal and penalties for moving toward danger. Eventually, it learns the safest and shortest path.

Reinforcement Learning - Learning by Doing:

Imagine you have a little robot, let's call it Robo, who loves exploring mazes. Robo's goal is to find a precious diamond hidden in the maze, but there's also dangerous fire blocking its way.

Reinforcement learning is like Robo learning by playing a game. It doesn't know the right path at first, so it tries different paths in the maze (trial and error). Whenever Robo takes a good step towards the diamond, it gets a treat (reward). But if Robo goes towards the fire, it gets a small scolding (punishment).

The more treats Robo gets, the happier it becomes. So, Robo learns from its mistakes and starts to figure out the best way to reach the diamond while avoiding the fire. It's like Robo is teaching itself to play the maze game better and better with each try!.

Categories of Reinforcement Learning

1. Positive Reinforcement

A reward is given to encourage correct behaviour.

Example: Giving a dog a treat when it sits on command.

2. Negative Reinforcement

An uncomfortable condition is removed to encourage correct behaviour.

Example: Stopping pressure when a dog sits, teaching it to sit to avoid discomfort.

Examples

- Game playing (Chess, Go)
- Robotics
- Self-driving cars

Concept Learning:

Defined as A task of acquiring a potential hypothesis (solution) that best fits the training examples.

Definition of Concept Learning

Concept learning involves inferring a boolean-valued function from training examples of its input and output.

Concept learning in machine learning is the process of teaching a machine to **learn a general rule or pattern from training examples**. Instead of memorizing data, the machine learns a **concept** that helps it identify or classify new, unseen data.

Example: By showing many images of cats, the machine learns the concept of a “cat” and can recognize new cat images later.

Types of Concept Learning

1. Supervised Concept Learning

- Uses **labelled data**.
- The machine learns by mapping inputs to known outputs.

Example: Images labelled as “cat” or “dog” help the model classify new images correctly.

2. Unsupervised Concept Learning

- Uses **unlabelled data**.
- The machine finds patterns or groups on its own.

Example: Grouping customers based on similar buying behavior without predefined labels.

Importance of Concept Learning

Concept learning helps machines:

- **Generalize knowledge** from data
- Make predictions in real-world applications

Concept Learning Working Process

1. **Learning from Examples** – The machine analyzes given data.
2. **Hypothesis Formation** – It creates a general rule to explain the data.
3. **Hypothesis Space** – It searches for the best possible rule.
4. **Generalization** – The learned concept is applied to new data.
5. **Refinement** – The concept is improved as more data is added

A CONCEPT LEARNING TASK

To ground the discussion, the text considers the task of learning the target concept "**days on which my friend Aldo enjoys his favorite water sport**".

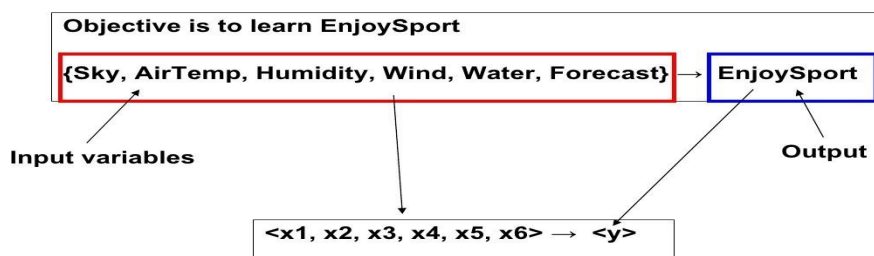
Table: Positive and Negative Training Examples

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

Objective is to learn EnjoySport

{ Sky, AirTemp, Humidity, Wind, Water, Forecast} $\rightarrow$ EnjoySport

Concept Learning Task



Notations

- **Given:**
 - Instances X : Possible days, each described by the attributes
 - *Sky* (with possible values *Sunny*, *Cloudy*, and *Rainy*),
 - *AirTemp* (with values *Warm* and *Cold*),
 - *Humidity* (with values *Normal* and *High*),
 - *Wind* (with values *Strong* and *Weak*),
 - *Water* (with values *Warm* and *Cool*), and
 - *Forecast* (with values *Same* and *Change*).
 - Hypotheses H : Each hypothesis is described by a conjunction of constraints on the attributes *Sky*, *AirTemp*, *Humidity*, *Wind*, *Water*, and *Forecast*. The constraints may be "?" (any value is acceptable), " $\emptyset$ " (no value is acceptable), or a specific value.
 - Target concept c : $EnjoySport : X \rightarrow \{0, 1\}$
 - Training examples D : Positive and negative examples of the target function (see Table 2.1).
- **Determine:**
 - A hypothesis h in H such that $h(x) = c(x)$ for all x in X .

Notations

	Instances (X)						Target Concept (C)
Example	<i>Sky</i>	<i>AirTemp</i>	<i>Humidity</i>	<i>Wind</i>	<i>Water</i>	<i>Forecast</i>	<i>EnjoySport</i>
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

Training examples (D)

Hypothesis: In machine learning, a **hypothesis** is an **initial assumption or guess** made to solve a problem based on available data. It is not proven at first and must be **tested using data**. A good hypothesis should be **testable** and give either true or false results.

In many cases, the terms **hypothesis** and **model** are used together.

However:

- A **hypothesis** is an assumption about the solution.
- A **model** is the mathematical representation used to test that hypothesis.

Example of Hypothesis: If a scientist assumes that **ultraviolet (UV) rays can damage eyes and may cause blindness** this statement is a **hypothesis** because it is an assumption that needs to be tested.

In supervised machine learning, the goal is to find a hypothesis that **best maps input data to output data** using labelled examples. The machine searches for the best hypothesis from a group of possible hypotheses.

A **hypothesis (h)** is a **single function or rule** selected from the hypothesis space that best fits the training data. It is used to make predictions on new data.

Mathematical Representation of Hypothesis

A simple hypothesis can be written as:

$$y = mx + c$$

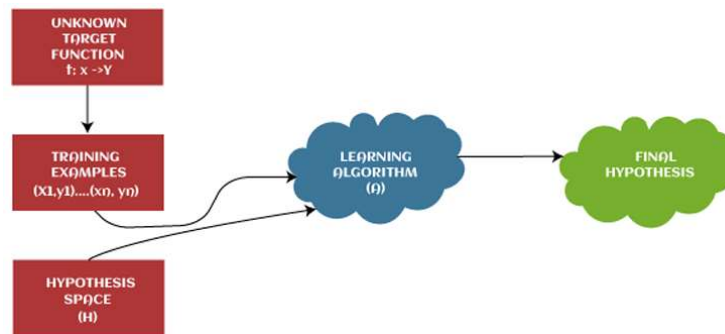
Where:

y → output (range)

x → input (domain)

m → slope (rate of change)

c → intercept (constant)



Hypothesis Space (H)

The **hypothesis space (H)** is the **set of all possible hypotheses** that a machine learning algorithm can choose from. It depends on:

- The problem definition
- The chosen model

Hypothesis Representation (Example)

A hypothesis is a vector of six constraints (Sky, AirTemp, Humidity, Wind, Water, and Forecast).

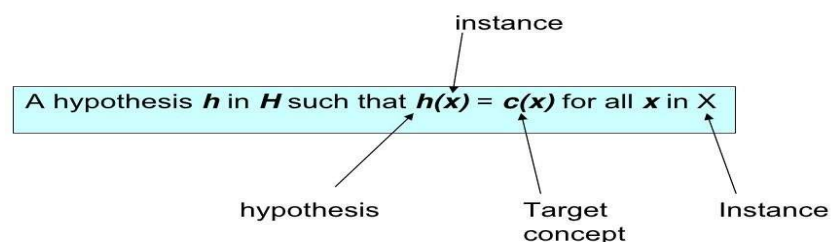
- "?" indicates any value is acceptable.
- **Required Value** (e.g., Warm) specifies a single value.
- "Ø" indicates no value is acceptable.

Hypothesis Extremes:

- **Most General:** (?, ?, ?, ?, ?, ?)
- **Most Specific:** (Ø, Ø, Ø, Ø, Ø, Ø)

Concept Learning

Acquiring the definition of a general category from a given set of positive and negative training examples of the category.



CONCEPT LEARNING AS SEARCH

Concept learning is viewed as searching through a large space of hypotheses defined by the representation.

- **Instance Space (X):** In the EnjoySport task, there are $3 \times 2 \times 2 \times 2 \times 2 \times 2 = 96$ distinct instances.
- **Hypothesis Space (H):** There are 5,120 syntactically distinct hypotheses, but only 973 semantically distinct ones.

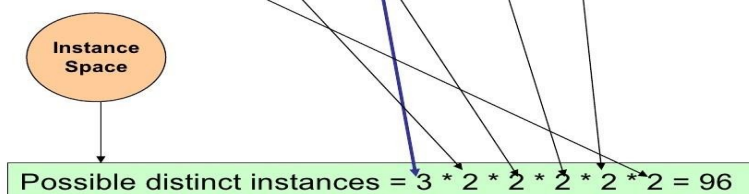
Instance Space

Suppose that the target hypothesis that we want to learn for the current problem is represented as a conjunction of all the attributes

Sky = **AND** *AirTemp* = **AND** *Humidity*= **AND** *Wind* = **AND** *Water*= **AND** *Forecast*= **THEN** *EnjoySport*=

Instance Space

Suppose the attribute **Sky** has three possible values, and that **AirTemp**, **Humidity**, **Wind**, **Water**, and **Forecast** each have two possible values



Hypothesis Space

Example	<i>Sky</i>	<i>AirTemp</i>	<i>Humidity</i>	<i>Wind</i>	<i>Water</i>	<i>Forecast</i>	<i>EnjoySport</i>
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes

$h = \langle \text{Sunny, Warm, ?, Strong, Warm, Same} \rangle$

Normal / High

Hypothesis Space

Hypothesis Space: A set of all possible hypotheses

Possible syntactically distinct Hypotheses for *EnjoySport*
 $= 5 * 4 * 4 * 4 * 4 * 4 = 5120$

- Sky has **three** possible values
- **Fourth** value don't care (?)
- **Fifth** value is empty set $\emptyset$

Hypothesis Space

Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1		Warm	Normal	Strong	Warm	Same	Yes
2		Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No

$h = \langle \emptyset, \text{Warm}, ?, \text{Strong}, \text{Warm}, \text{Same} \rangle$

Concept learning as search:

Concept learning can be viewed as the task of searching through a large space of hypothesis implicitly defined by the hypothesis representation.

The goal of concept learning search is to find the hypothesis that best fits the training examples.

General-to-Specific Learning

Every day Tom his
enjoy i.e., Only
positive examples.

Most General Hypothesis: $h = \langle ?, ?, ?, ?, ?, ? \rangle$

Most Specific Hypothesis: $h = \langle \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset \rangle$

Definition

Given hypotheses **hj** and **hk**, **hj** is **more_general_than_or_equal_to** **hk** if and only if any instance that satisfies **hk** also satisfies **hj**.

Definition: Let h_j and h_k be boolean-valued functions defined over X . Then h_j is **more_general_than_or_equal_to** h_k (written $h_j \geq_g h_k$) if and only if

$$(\forall x \in X)[(h_k(x) = 1) \rightarrow (h_j(x) = 1)]$$

We can also say that **hj** is **more_specific_than** **hk** when **hk** is **more_general_than** **hj**.

FIND-S: FINDING A MAXIMALLY SPECIFIC HYPOTHESIS

The FIND-S algorithm searches from the most specific hypothesis toward increasingly general ones.

FIND-S Algorithm:

1. Initialize h to the most specific hypothesis in H .
2. For each positive training instance x :
 - For each attribute constraint a_i in h :
 - If a_i is satisfied by x , do nothing.
 - Else, replace a_i in h by the next more general constraint satisfied by x .
3. Output hypothesis h .

FIND-S ignores negative examples. It is guaranteed to find the most specific hypothesis consistent with positive examples, provided the target concept is in $\mathcal{H}$ and data is error-free.

Example:

Step 1: FIND-S

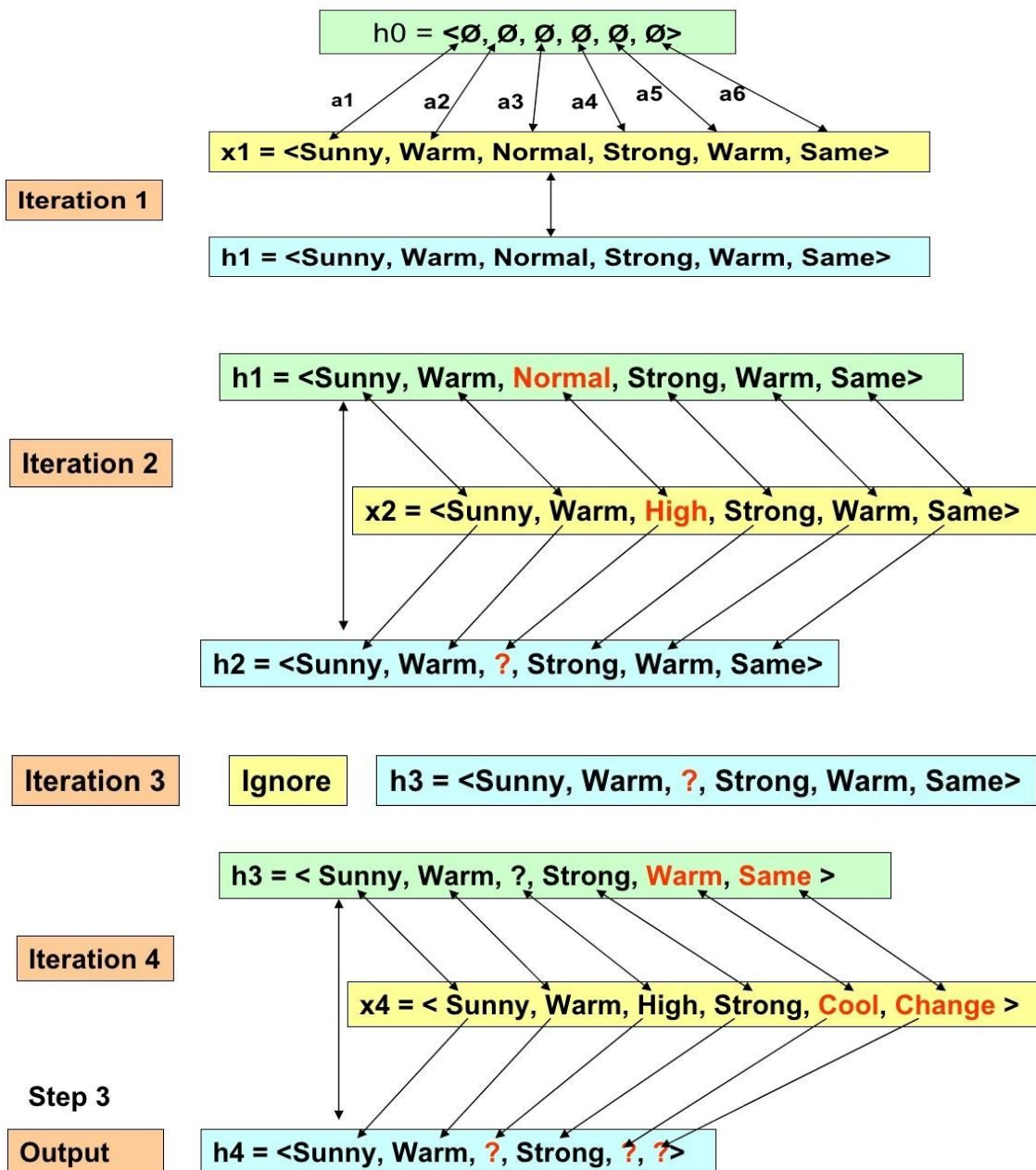
Example	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

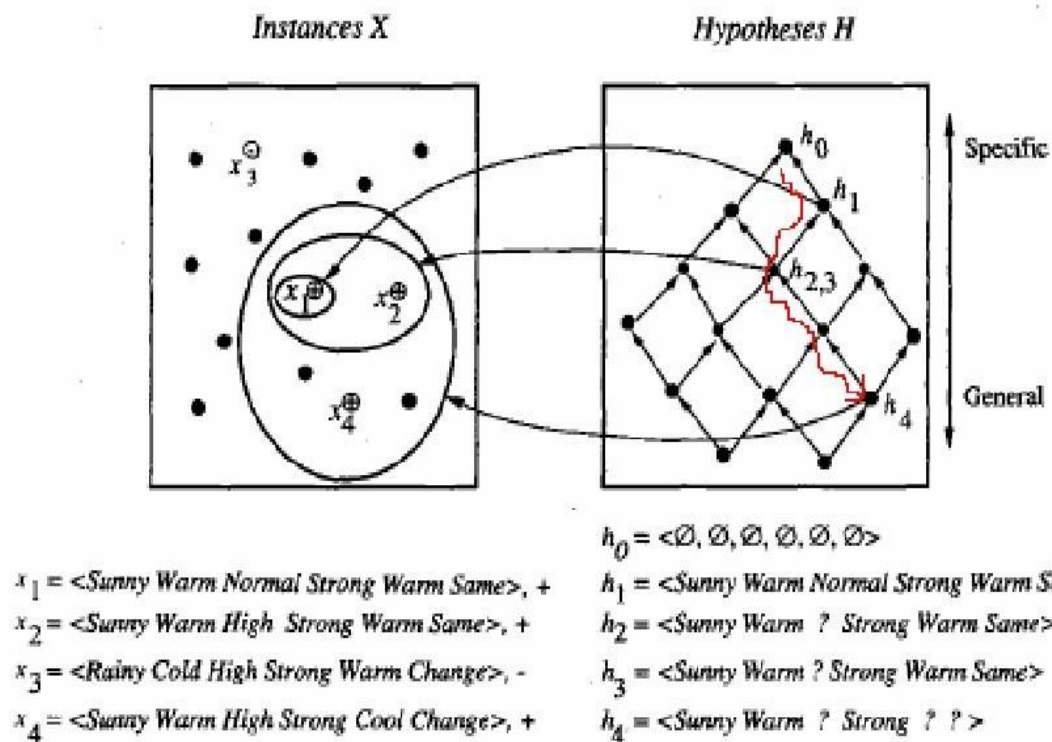
1. Initialize h to the most specific hypothesis in H

$h_0 = \langle \emptyset, \emptyset, \emptyset, \emptyset, \emptyset, \emptyset \rangle$

Step 2: FIND-S

2. For each positive training instance x
 - For each attribute constraint a_i in h
 - If the constraint a_i is satisfied by x
 - Then do nothing
 - Else replace a_i in h by the next more general constraint that is satisfied by x





Version Space

A **Version Space** is a **structured way of representing knowledge** that keeps track of **all possible valid hypotheses** learned from training examples, without storing the examples themselves.

It is used in **concept learning** to manage multiple possible models at the same time.

A hypothesis is considered **plausible** if it:

- Covers all **positive examples**
- Excludes all **negative examples**

Key Characteristics of Version Space

- Represents **tentative hypotheses (heuristics)**
- Contains **all possible valid descriptions** of a concept
- Works only with **discrete attribute values**
- Each hypothesis must satisfy all observed examples

Structure of Version Space

A Version Space consists of **two complementary trees**:

1. **Generalization Tree**

- Contains **overly general hypotheses**
- Each node represents a model that matches **all instances** in its subtree
- Edges show **generalization relationships**

2. Specialization Tree

- Contains **overly specific hypotheses**
- Each node represents a model that matches **only specific instances**
- Edges show **specialization relationships**

Candidate Elimination Algorithm (CEA)

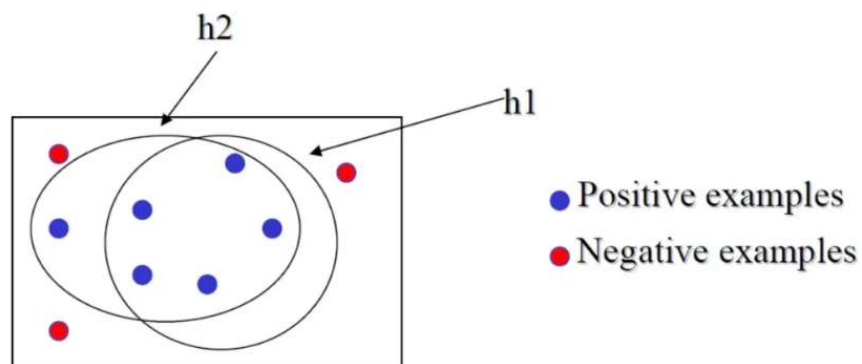
The **Candidate Elimination Algorithm (CEA)** is a **concept learning algorithm** used in **supervised machine learning**. It finds **all possible hypotheses** that are consistent with the **training data**, known as the **Version Space**.

CEA maintains **two boundaries**:

- **S (Specific boundary)** – the most specific hypotheses
- **G (General boundary)** – the most general hypotheses

As training examples are processed, CEA **eliminates inconsistent hypotheses** and gradually **narrows the version space** to identify the best possible concept.

Why Do We Need the Candidate Elimination Algorithm?



In concept learning, the machine starts with **no prior knowledge** about the target concept. At this stage, the model may make **wrong assumptions**, leading to **overfitting** or **underfitting**.

If we keep only:

- **A very specific hypothesis**, the model may overfit and fail on new data.
- **A very general hypothesis**, the model may underfit and give inaccurate results.

The **Candidate Elimination Algorithm (CEA)** solves this problem by maintaining **both**:

- **General boundary (G)** and **Specific boundary (S)** instead of relying on a single hypothesis.

Candidate Elimination Algorithm (CEA):

1. Initialize

- Set **S** to the most specific hypothesis in **H**
- Set **G** to the most general hypothesis in **H**

2. For each training example **d** in the dataset:

a) If **d** is a positive example:

- Remove from **G** all hypotheses that do not cover **d**
- For each hypothesis **s** in **S** that does not cover **d**:
 - Remove **s** from **S**
 - Add to **S** all **minimal generalizations** of **s** that cover **d**
- Remove from **S** any hypothesis that is more general than another hypothesis in **S**

b) If **d** is a negative example:

- Remove from **S** all hypotheses that cover **d**
- For each hypothesis **g** in **G** that covers **d**:
 - Remove **g** from **G**
 - Add to **G** all **minimal specializations** of **g** that exclude **d**
- Remove from **G** any hypothesis i.e., less general than another hypothesis in **G**

3. Repeat

- Continue until all training examples are processed

4. Result

- The **Version Space** consists of all hypotheses between **S** and **G**

In order to this algorithm the inputs and outputs are as follows:

Input:

- Training dataset with positive and negative examples
- Hypothesis space **H**

Output:

- Version Space (set of all consistent hypotheses)

Candidate Elimination Algorithm Example:

Problem Statement: “When should a student be allowed into the library?”

Attribute	Possible Values
ID Card	Yes, No
Time	Morning, Evening
Bag	Allowed, Not Allowed
Silence	Yes, No

Training Dataset

Example	ID Card	Time	Bag	Silence	Allowed?
1	Yes	Morning	Allowed	Yes	Yes
2	Yes	Evening	Allowed	Yes	Yes
3	No	Morning	Allowed	Yes	No
4	Yes	Morning	Allowed	No	No

Step 1: Initialization

- **Most Specific Hypothesis (S):**

$$S = \langle \emptyset, \emptyset, \emptyset, \emptyset \rangle$$

- **Most General Hypothesis (G):**

$$G = \langle ?, ?, ?, ? \rangle$$

Step 2: Process Example 1 (Positive)

$$\{\text{Yes, Morning, Allowed, Yes}\} \rightarrow \text{Yes}$$

- Generalize **S** to match this example

$$S = \langle \text{Yes, Morning, Allowed, Yes} \rangle$$

- **G remains unchanged**

Step 3: Process Example 2 (Positive)

$$\langle \text{Yes, Evening, Allowed, Yes} \rangle \rightarrow \text{Yes}$$

- Time differs $\rightarrow$ generalize **S**

$$S = \langle \text{Yes, ?, Allowed, Yes} \rangle$$

- **G remains unchanged**

Step 4: Process Example 3 (Negative)

<No, Morning, Allowed, Yes> → No

- This example matches **G**, so we must **specialize G**

Possible minimal specializations of G that exclude this example and still cover S:

G = <Yes, ?, ?, ?>,

<?, ?, ?, ?> ✗ (removed)

After pruning:

G = <Yes, ?, ?, ?>

Step 5: Process Example 4 (Negative)

<Yes, Morning, Allowed, No> → No

- This matches **G**, so specialize again

Possible specializations:

G = <Yes, ?, ?, Yes>

- **S remains unchanged**

Final Result

Specific Boundary (S): S = <Yes, ?, Allowed, Yes>

General Boundary (G): G = <Yes, ?, ?, Yes>

Inductive Bias:

Definition: Inductive Bias is the set of assumptions that a machine learning algorithm makes about the relationship between input features and output labels in order to generalize from training data to unseen data.

In simple Inductive bias is the prior knowledge that helps a machine learn patterns and make predictions on new data.

Why Inductive Bias is Necessary

- Training data is limited
- Many possible models can fit the same data
- Without assumptions, the model cannot choose the best one

☞ Inductive bias allows the algorithm to learn efficiently and predict accurately on new data.

Types of Inductive Bias

Inductive bias is mainly classified into two types:

Restrictive Bias: Restrictive bias limits the type of functions the algorithm can learn.

Examples:

- Linear Regression assumes a linear relationship
- Decision Trees assume the concept can be represented in a tree structure

Effect:

- Faster learning
- Less flexibility
- May fail for complex relationships

Preferential Bias: Preferential bias prefers certain models over others, even when multiple models fit the data.

Examples:

- Neural networks prefer complex, non-linear functions
- k-Nearest Neighbors (k-NN) assumes similar inputs give similar outputs

Effect:

- More flexibility
- Can model complex patterns

Importance of Inductive Bias

Inductive bias directly affects model performance and generalization.

- Good inductive bias → good performance on new data
- Bad inductive bias → overfitting or underfitting

Example:

- Using linear regression for a non-linear problem → poor results
- Using a very complex model for simple data → overfitting

Choosing the Right Inductive Bias

To choose the correct inductive bias:

1. Start with a simple model
Helps understand data structure

2. Evaluate model performance
Check results on validation data
3. Consider problem complexity
Use complex models only when needed
4. Consider data size
 - Small data → simple models
 - Large data → complex models

Common Problems and Solutions

Problem 1: Poor Validation Performance

- Try a different algorithm
- Change inductive bias
- Adjust model complexity

Problem 2: Overfitting

- Use a simpler model
- Apply regularization (L1, L2)
- Use techniques like dropout

Problem 3: Underfitting

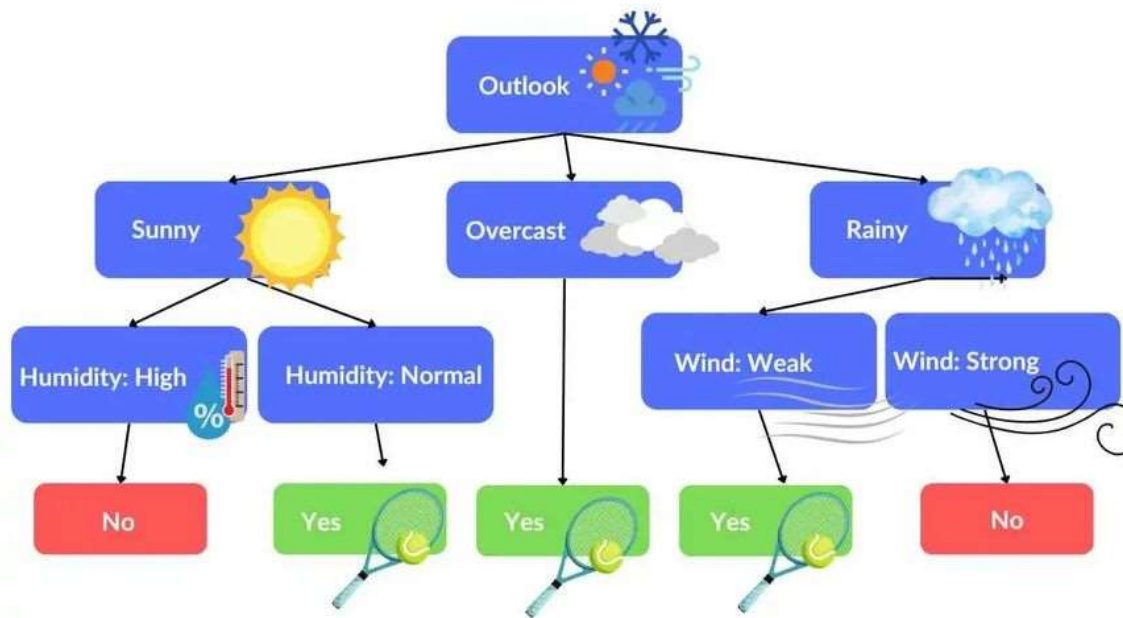
- Increase model complexity
- Use richer features
- Change inductive bias

Decision Tree learning:

A **Decision Tree** is a **supervised machine learning algorithm** that is mainly used for **classification** and also for **regression** problems. It works like a **flowchart**, where decisions are made step by step based on input features.

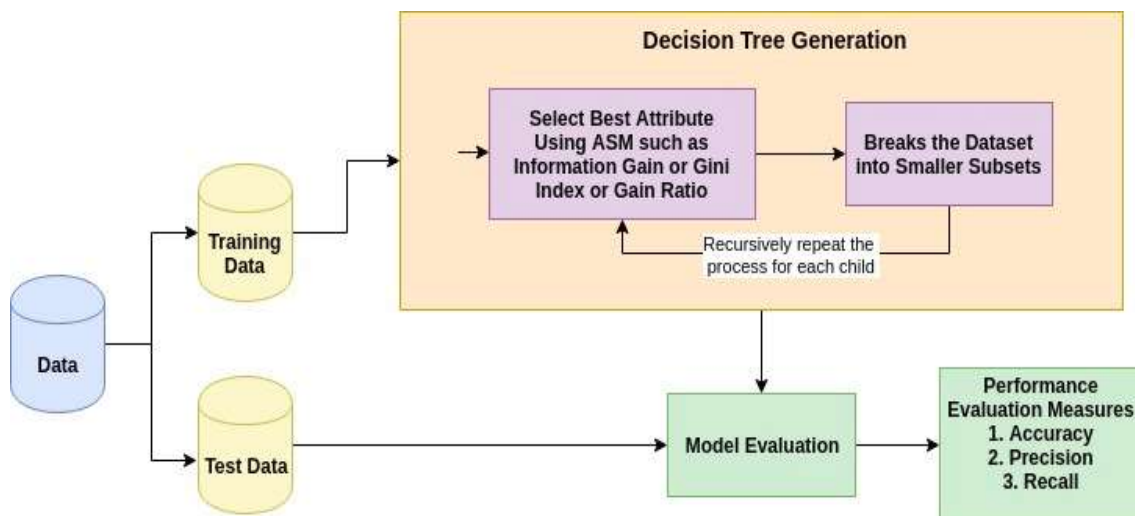
- It has a **pre-defined target variable**
- It works with **categorical and continuous** input variables
- It splits data into **homogeneous groups**

In simple A decision tree predicts the output by asking a series of questions and following the answers to reach a final decision.



Decision Trees functionality

A **Decision Tree** works by **recursively splitting** a dataset into smaller subsets based on the **most informative features**. This process continues until the data becomes sufficiently **pure** or a **stopping condition** is reached.



Step 1: Data Collection

- The process starts with the **dataset**.
- This dataset contains **input features** and a **target variable**.

Step 2: Split Data

- The data is divided into:
 - **Training Data** – used to build the decision tree

- **Test Data** – used to evaluate the model

Step 3: Decision Tree Generation (Training Phase)

Step 3.1: Select Best Attribute

- From the training data, the algorithm selects the **best attribute** for splitting.
- This is done using **Attribute Selection Measures (ASM)** such as:
 - **Information Gain**
 - **Gini Index**
 - **Gain Ratio**

These measures help identify the attribute that creates the **most homogeneous subsets**.

Step 3.2: Split the Dataset

- The dataset is split into **smaller subsets** based on the selected attribute.
- Each subset becomes a **child node**.

Step 3.3: Recursive Splitting

- The same process is **repeated recursively** for each child node:
 - Select the best attribute
 - Split the data
- This continues until:
 - Data becomes pure, or
 - Stopping criteria are met

Step 4: Tree Completion

- When no further meaningful splits are possible, the nodes become **leaf nodes**.
- Leaf nodes store the **final decision or prediction**.

Step 5: Model Evaluation

- The trained decision tree is tested using **test data**.
- Predictions are compared with actual values.

Step 6: Performance Evaluation Measures

The performance of the decision tree is measured using:

1. **Accuracy** – Correct predictions out of total predictions
2. **Precision** – Correct positive predictions

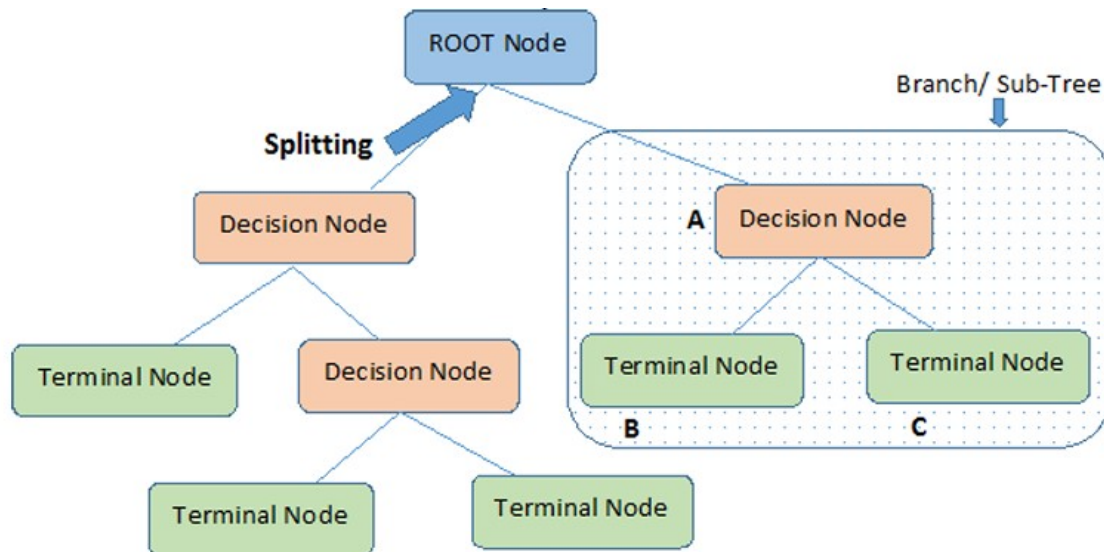
3. Recall – Ability to find all positive cases

Final Flow

Data → Training Data → Select Best Attribute → Split Data → Repeat Recursively → Leaf Nodes → Test Data → Model Evaluation → Performance Metrics

Decision Tree Structure Terminology:

A **Decision Tree** is organized in a hierarchical structure consisting of **nodes and branches**. Each part of the tree has a specific role.



1. Root Node

- The **topmost node** of the tree
- Represents the **entire dataset**
- The **first split** happens here
- Chosen using a splitting criterion (Information Gain / Gini Index)

The node labelled **ROOT Node** is the starting point.

2. Splitting

- **Splitting** is the process of dividing a node into two or more sub-nodes
 - It is based on the value of an attribute
- The arrow labelled **Splitting** shows the root node dividing into decision nodes.

3. Decision Node

- A node that **tests an attribute**
- It can further split into:

- More decision nodes, or
- Terminal (leaf) nodes
- Nodes labelled **Decision Node**
- Node **A** is a decision node inside a **sub-tree**

4. Branch / Sub-Tree

- A **branch** represents the outcome of a decision
- A **sub-tree** is a part of the tree that behaves like a smaller decision tree
- The dotted box represents a **Branch / Sub-Tree**
- Node **A** with its children forms a sub-tree

5. Terminal Node (Leaf Node)

- Nodes where **splitting stops**
- Represents the **final decision or prediction**
- No further child nodes
- Nodes labelled **Terminal Node**
- Nodes **B** and **C** are terminal nodes

6. Parent and Child Nodes

- A **parent node** splits into one or more **child nodes**
- Relationship shows the flow of decisions
- Root node is the parent of decision nodes
- Decision nodes are parents of terminal nodes

Types of Decision Trees

The type of a decision tree depends on the **type of target (output) variable**. Based on this, decision trees are classified into **two types**:

1. Categorical Variable Decision Tree (Classification Tree)

A decision tree is called a **Categorical Variable Decision Tree** when the **target variable is categorical** (discrete values).

Characteristics

- Output is a **class label**
- Used for **classification problems**
- Answers are usually **Yes/No, True/False**, or class names

Example

In the student problem:

- Target variable: **“Student will play cricket or not”**
- Possible values: **YES / NO**

Since the output is categorical, this is a **categorical (classification) decision tree**.

2. Continuous Variable Decision Tree (Regression Tree)

A decision tree is called a **Continuous Variable Decision Tree** when the **target variable is continuous (numeric)**.

Characteristics

- Output is a **numerical value**
- Used for **regression problems**
- Predicts quantities such as income, price, or amount

Example

- Problem: Predict a customer's **income**
- Income is important for deciding whether the customer will pay the **insurance renewal premium**
- Direct income data is not available
- So, a decision tree is built using attributes like:
 - Occupation
 - Product type
 - Customer behaviour

The tree predicts a **numeric income value**, so this is a **continuous (regression) decision tree**

How Does a Decision Tree Decide Where to Split

The accuracy of a decision tree highly depends on **how and where it splits the data**. Decision trees split data in such a way that the resulting sub-nodes become **more homogeneous (pure)** with respect to the target variable.

Different **splitting algorithms** are used based on whether the target variable is **categorical or continuous**.

Basic Idea of Splitting

- At each node, the decision tree:
 - Tries **all available input variables**

- Evaluates all possible splits
- Selects the split that produces the **most homogeneous sub-nodes**
- A node is considered **pure** if most of its data points belong to the same class.

Splitting Algorithms in Decision Trees

Decision trees use the following commonly used algorithms:

1. Gini Index

The **Gini Index** measures the **purity** of a node. If a node is pure, the probability of randomly selecting two items of the same class is high.

- Used for **categorical target variables** (Yes/No, Success/Failure)
- Performs **binary splits only**
- Higher Gini value → higher homogeneity (as per your notes)
- Used by **CART (Classification and Regression Tree)** algorithm

Gini Formula

$$Gini = p^2 + q^2$$

Where:

- p = probability of success
- q = probability of failure

Steps to Calculate Gini for a Split:

1. Calculate Gini for each sub-node
2. Calculate **weighted Gini** for the split

Example: Gini Index Calculation

We want to predict whether **employees will get a promotion (Yes / No)** based on input variables:

- **Department**
- **Experience Level**

We use the **Gini Index** to decide the best split.

Given Data

- Total employees = **40**
- Employees promoted = **22**
- Employees not promoted = **18**

Split 1: Based on Department**Technical Department**

- Total employees = **25**
- Promoted = **18 (72%)**
- Not Promoted = **7 (28%)**

Gini (Technical)

$$Gini = (0.72)^2 + (0.28)^2 = 0.5184 + 0.0784 = 0.5968$$

Non-Technical Department

- Total employees = **15**
- Promoted = **4 (27%)**
- Not Promoted = **11 (73%)**

Gini (Non-Technical)

$$Gini = (0.27)^2 + (0.73)^2 = 0.0729 + 0.5329 = 0.6058$$

Weighted Gini for Department Split

$$\begin{aligned} Gini_{Department} &= \frac{25}{40} \times 0.5968 + \frac{15}{40} \times 0.6058 \\ &= 0.373 + 0.227 = 0.60 \end{aligned}$$

Split 2: Based on Experience Level**Experience < 5 Years**

- Total employees = **18**
- Promoted = **6 (33%)**
- Not Promoted = **12 (67%)**

Gini (Low Experience)

$$Gini = (0.33)^2 + (0.67)^2 = 0.1089 + 0.4489 = 0.5578$$

Experience $\geq$ 5 Years

- Total employees = **22**
- Promoted = **16 (73%)**
- Not Promoted = **6 (27%)**

Gini (High Experience)

$$Gini = (0.73)^2 + (0.27)^2 = 0.5329 + 0.0729 = 0.6058$$

Weighted Gini for Experience Split

$$Gini_{Experience} = \frac{18}{40} \times 0.5578 + \frac{22}{40} \times 0.6058$$

$$= 0.251 + 0.333 = 0.584$$

Final Comparison

Split Variable	Weighted Gini
Department	0.60
Experience	0.584

Conclusion

- The **Experience split** has a **lower weighted Gini value**.
- Lower Gini → better purity.
- The decision tree will split the node on **Experience Level**.

2. Chi-Square Algorithm

Chi-square measures the **statistical significance** of the difference between:

- Parent node and
- Resulting sub-nodes

Key Points

- Used for **categorical target variables**
- Allows **multi-way splits**
- Higher Chi-square value → more significant split
- Generates **CHAID (Chi-square Automatic Interaction Detector)** tree

Chi-Square Formula

$$\chi^2 = \sum \frac{(Actual - Expected)^2}{Expected}$$

Steps to Calculate Chi-Square

1. Calculate actual and expected values
2. Compute deviation (Actual – Expected)

3. Calculate Chi-square for each class
4. Sum Chi-square values for the split

Result

Chi-square also identifies **Gender** as the best split over **Class**.

3. Information Gain (Entropy Based)

Information Gain is based on **Entropy**, which measures **disorder or impurity** in a node.

- Pure node → low entropy
- Impure node → high entropy

Entropy Properties

- Entropy = 0 → completely pure
- Entropy = 1 → maximum impurity (50%–50%)

Entropy Formula

$$Entropy = -p \log_2 p - q \log_2 q$$

Steps to Calculate Information Gain

1. Calculate entropy of parent node
2. Calculate entropy of each child node
3. Compute weighted entropy of split
4. **Information Gain = Parent Entropy – Split Entropy**

Example: Information Gain in Decision Tree

We want to predict whether a **customer will buy a product (Yes / No)** based on the input variable:

- **Membership Type (Premium / Regular)**

We use **Information Gain** to decide the best split.

Given Data

- Total customers = **20**
- Buy = **12**
- Not Buy = **8**

Step 1: Entropy of Parent Node

$$Entropy(parent) = -\frac{12}{20} \log_2 \frac{12}{20} - \frac{8}{20} \log_2 \frac{8}{20}$$

$$\begin{aligned}
 &= -(0.6)\log_2(0.6) - (0.4)\log_2(0.4) \\
 &= 0.442 + 0.529 = 0.971
 \end{aligned}$$

Step 2: Split on Membership Type

Premium Customers

- Total = 10
- Buy = 9
- Not Buy = 1

$$\begin{aligned}
 Entropy(Premium) &= -(0.9)\log_2(0.9) - (0.1)\log_2(0.1) \\
 &= 0.136 + 0.332 = 0.468
 \end{aligned}$$

Regular Customers

- Total = 10
- Buy = 3
- Not Buy = 7

$$\begin{aligned}
 Entropy(Regular) &= -(0.3)\log_2(0.3) - (0.7)\log_2(0.7) \\
 &= 0.521 + 0.360 = 0.881
 \end{aligned}$$

Step 3: Weighted Entropy of Split

$$\begin{aligned}
 Entropy(split) &= \frac{10}{20} \times 0.468 + \frac{10}{20} \times 0.881 \\
 &= 0.234 + 0.441 = 0.675
 \end{aligned}$$

Step 4: Information Gain

$$\begin{aligned}
 InformationGain &= Entropy(parent) - Entropy(split) \\
 &= 0.971 - 0.675 = 0.296
 \end{aligned}$$

Final Result

- **Information Gain = 0.296**
- This split significantly reduces entropy.

Conclusion: Since the split on Membership Type results in high information gain, the decision tree will split on Membership Type.

4. Reduction in Variance (For Regression Trees)

Reduction in Variance is used when the **target variable is continuous**.

The split that results in the **lowest variance** is chosen.

Variance Formula

$$Variance = \frac{\sum(X - \bar{X})^2}{n}$$

Where:

- X = actual value
- $\bar{X}$ = mean
- n = number of values

Steps to Calculate Variance

1. Calculate variance of parent node
2. Calculate variance of each sub-node
3. Compute weighted variance for split
4. Choose split with **minimum variance**

Example: Reduction in Variance:

We want to decide the best split to predict whether an **employee gets a bonus**.

Assign:

- **1** → **Bonus Given**
- **0** → **No Bonus**

We compare splits based on:

- **Department**
- **Experience Level**

Given Data (Root Node)

- Total employees = **40**
- Bonus Given = **20**
- No Bonus = **20**

Step 1: Variance at Root Node

Mean of Root Node

$$Mean = \frac{(20 \times 1) + (20 \times 0)}{40} = 0.5$$

Variance at Root

$$Variance = \frac{20(1 - 0.5)^2 + 20(0 - 0.5)^2}{40}$$

$$\begin{aligned}
 &= \frac{20(0.25) + 20(0.25)}{40} \\
 &= \frac{10}{40} = 0.25
 \end{aligned}$$

Split 1: Based on Department

Technical Department

- Total = **25**
- Bonus = **18**
- No Bonus = **7**

Mean

$$Mean = \frac{18}{25} = 0.72$$

Variance

$$\begin{aligned}
 Variance &= \frac{18(1 - 0.72)^2 + 7(0 - 0.72)^2}{25} \\
 &= \frac{18(0.0784) + 7(0.5184)}{25} \\
 &= \frac{1.41 + 3.63}{25} = 0.20
 \end{aligned}$$

Non-Technical Department

- Total = **15**
- Bonus = **2**
- No Bonus = **13**

Mean

$$Mean = \frac{2}{15} = 0.13$$

Variance

$$\begin{aligned}
 Variance &= \frac{2(1 - 0.13)^2 + 13(0 - 0.13)^2}{15} \\
 &= \frac{2(0.7569) + 13(0.0169)}{15} \\
 &= \frac{1.51 + 0.22}{15} = 0.115
 \end{aligned}$$

Weighted Variance for Department Split

$$\begin{aligned} \text{Variance}_{Dept} &= \frac{25}{40} \times 0.20 + \frac{15}{40} \times 0.115 \\ &= 0.125 + 0.043 = 0.168 \end{aligned}$$

Split 2: Based on Experience Level

Experience < 5 Years

- Total = **18**
- Bonus = **6**
- No Bonus = **12**

Mean

$$\text{Mean} = \frac{6}{18} = 0.33$$

Variance

$$\begin{aligned} \text{Variance} &= \frac{6(1 - 0.33)^2 + 12(0 - 0.33)^2}{18} \\ &= \frac{6(0.4489) + 12(0.1089)}{18} \\ &= \frac{2.69 + 1.31}{18} = 0.22 \end{aligned}$$

Experience ≥ 5 Years

- Total = **22**
- Bonus = **14**
- No Bonus = **8**

Mean

$$\text{Mean} = \frac{14}{22} = 0.64$$

Variance

$$\begin{aligned} \text{Variance} &= \frac{14(1 - 0.64)^2 + 8(0 - 0.64)^2}{22} \\ &= \frac{14(0.1296) + 8(0.4096)}{22} \\ &= \frac{1.81 + 3.28}{22} = 0.23 \end{aligned}$$

Weighted Variance for Experience Split

$$\begin{aligned}
 \text{Variance}_{Exp} &= \frac{18}{40} \times 0.22 + \frac{22}{40} \times 0.23 \\
 &= 0.099 + 0.126 = 0.225
 \end{aligned}$$

Final Comparison

Split Variable	Weighted Variance
Department	0.168
Experience	0.225

Conclusion

- **Department split** has **lower variance**
- Lower variance → better prediction

Final Decision

The decision tree will split on **Department**.